

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Convertir un nombre décimal en binaire et vice versa
- Convertir un nombre binaire en hexadécimal et vice versa
- Expliquer la différence entre ASCII et Unicode/UTF-8
- Coder un caractère à partir de son code ASCII ou Unicode
- Calculer la taille d'une image bitmap (pixels, résolution, profondeur de couleur)
- Appliquer ces notions à des exemples concrets pour l'enseignement des langues

Plan du cours

1. Introduction : pourquoi représenter l'information ? (5 min)
2. Le système binaire (base 2) (20 min)
3. Le système hexadécimal (base 16) (15 min)
4. Codage des caractères : ASCII, Unicode, UTF-8 (20 min)
5. Codage des images : bitmap, pixels, résolution, couleur (20 min)
6. Applications pédagogiques pour l'enseignement des langues (10 min)

1. Arbre des codages

```
graph TD
  INFO["Information numérique"] --> NOMBRES["Nombres"]
  INFO --> TEXTES["Textes"]
  INFO --> IMAGES["Images"]
  INFO --> SON["Son (échantillonnage)"]

  NOMBRES --> BIN["Binaire (base 2)<br/>0 et 1"]
  NOMBRES --> DEC["Décimal (base 10)<br/>0-9"]
  NOMBRES --> HEX["Hexadécimal (base 16)<br/>0-9, A-F"]

  TEXTES --> ASCII["ASCII<br/>128 car. (7 bits)"]
  TEXTES --> UNICODE["Unicode<br/>~150 000 car."]
  UNICODE --> UTF8["UTF-8<br/>1-4 octets"]

  IMAGES --> BITMAP["Bitmap<br/>Matrice de pixels"]
  BITMAP --> COULEUR["Couleur RVB<br/>24 bits/pixel"]
  BITMAP --> NB["Noir & Blanc<br/>1 bit/pixel"]

  SON --> ECH["Échantillonnage<br/>kHz + quantification"]

  style BIN fill:#e74c3c,color:#fff
```

```

style HEX fill:#e67e22,color:#fff
style ASCII fill:#3498db,color:#fff
style UTF8 fill:#2ecc71,color:#fff
style BITMAP fill:#9b59b6,color:#fff

```

L'information numérique est toujours représentée avec des 0 et des 1 (bits). Tout type de donnée (nombre, texte, image, son, vidéo) est converti en binaire pour être traité par un ordinateur.

2. Le système binaire (base 2)

Principe

Le système binaire utilise uniquement deux chiffres : 0 et 1. Chaque position correspond à une puissance de 2.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

Conversion décimal → binaire : méthode des divisions successives

Exemple : convertir 42 en binaire

```

42 ÷ 2 = 21, reste 0 (bit de poids faible)
21 ÷ 2 = 10, reste 1
10 ÷ 2 = 5, reste 0
5 ÷ 2 = 2, reste 1
2 ÷ 2 = 1, reste 0
1 ÷ 2 = 0, reste 1

```

Lecture des restes de bas en haut : $42_{10} = 101010_2$

Conversion binaire → décimal : méthode des puissances

```

1010102 = 1×25 + 0×24 + 1×23 + 0×22 + 1×21 + 0×20
= 32 + 0 + 8 + 0 + 2 + 0
= 4210

```

```

graph LR
    subgraph "Binaire → Décimal"
        B1["1010102"] --> P1["1×25 + 0×24 + 1×23 + 0×22 + 1×21 + 0×20"]
        P1 --> R1["= 32 + 0 + 8 + 0 + 2 + 0 = 4210"]
    end

```

```

subgraph "Décimal → Binaire"
    DEC1["4210"] --> DIV["42÷2=21 r1<br/>21÷2=10 r0<br/>10÷2=5 r0<br/>5÷2=2 r1<br/>2÷2=1 r0<br/>1÷2=0 r1"]
    DIV --> BIN1["1010102"]
end

```

```

style R1 fill:#2ecc71,color:#fff
style BIN1 fill:#2ecc71,color:#fff

```

Bit = Binary Digit. Un octet (byte) = 8 bits. Un octet permet de coder 256 valeurs différentes (2^8).

3. Le système hexadécimal (base 16)

Principe

L'hexadécimal utilise 16 chiffres : 0-9 puis A-B-C-D-E-F (A=10, B=11, C=12, D=13, E=14, F=15).

Pourquoi l'hexadécimal ? Parce qu'un chiffre hexa correspond exactement à 4 bits (un quartet), ce qui rend les conversions binaire-hexa très simples.

Tableau de conversion

```

block-beta
columns 4
block:group1
columns 4
B0["0000"] B1["0001"] B2["0010"] B3["0011"]
B4["0100"] B5["0101"] B6["0110"] B7["0111"]
B8["1000"] B9["1001"] BA["1010"] BB["1011"]
BC["1100"] BD["1101"] BE["1110"] BF["1111"]
end
space
block:group2
columns 4
H0["0"] H1["1"] H2["2"] H3["3"]
H4["4"] H5["5"] H6["6"] H7["7"]
H8["8"] H9["9"] HA["A"] HB["B"]
HC["C"] HD["D"] HE["E"] HF["F"]
end
space
block:group3
columns 4
D0["0"] D1["1"] D2["2"] D3["3"]
D4["4"] D5["5"] D6["6"] D7["7"]
D8["8"] D9["9"] D10["10"] D11["11"]
D12["12"] D13["13"] D14["14"] D15["15"]
end
group1 --> group2 --> group3

```

Conversion binaire → hexa : Grouper par 4 bits à partir de la droite, convertir chaque groupe.

1010 1110 0101₂ → A E 5₁₆ → AE5₁₆

A (1010) = 10₁₀
 E (1110) = 14₁₀
 5 (0101) = 5₁₀

Conversion hexa → binaire : Chaque chiffre hexa donne 4 bits.

3F₁₆ → 0011 1111₂ → 111111₂

Conversion hexa → décimal :

AE5₁₆ = A×16² + E×16¹ + 5×16⁰
 = 10×256 + 14×16 + 5
 = 2560 + 224 + 5
 = 2789₁₀

Usage courant : L'hexadécimal est utilisé pour représenter les couleurs en HTML/CSS (#FF0000 pour le rouge, #FFFFFF pour le blanc, #000000 pour le noir), les adresses mémoire, et les adresses MAC.

4. Codage des caractères

ASCII (American Standard Code for Information Interchange)

- Créé en 1963, standardisé en 1967
- 7 bits par caractère → 128 caractères (0-127)
- Contient : lettres anglaises (A-Z, a-z), chiffres (0-9), signes de ponctuation, caractères de contrôle

```
graph TD
  subgraph "Table ASCII (extrait)"
    L1["65 → A<br/>66 → B<br/>67 → C"]
    L2["97 → a<br/>98 → b<br/>99 → c"]
    L3["48 → 0<br/>49 → 1<br/>50 → 2"]
    L4["32 → espace<br/>33 → !<br/>63 → ?"]
  end

  subgraph "Limitation"
    LIM["Pas d'accents<br/>Pas de ç, é, è, ê, œ, æ<br/>Pas d'arabe, chinois, etc."]
  end

  L1 --> LIM
  L2 --> LIM
  L3 --> LIM
  L4 --> LIM
```

Exemple : Le mot "Hello" en ASCII :

```
H → 72 → 0100 1000
e → 101 → 0110 0101
l → 108 → 0110 1100
l → 108 → 0110 1100
o → 111 → 0110 1111
```

Unicode et UTF-8

Unicode : Répertoire universel de caractères. Chaque caractère possède un point de code unique (ex: U+0041 pour 'A', U+00E9 pour 'é', U+0628 pour 'ب').

UTF-8 : Encodage dominant sur le Web (> 95%). Utilise de 1 à 4 octets par caractère :

- 1 octet : caractères ASCII (U+0000 à U+007F) — compatibilité ascendante
- 2 octets : lettres accentuées, alphabets grec, cyrillique, arabe (U+0080 à U+07FF)
- 3 octets : caractères asiatiques (CJK), symboles divers (U+0800 à U+FFFF)
- 4 octets : caractères rares, émojis (U+10000 à U+10FFFF)

Exemples : A (U+0041, 1 octet 41), é (U+00E9, 2 octets C3 A9), ç (U+00E7, C3 A7), œ (U+0153, C5 93), ب (U+0628, D8 A8), ☺ (U+1F600, 4 octets F0 9F 98 80).

Application pour l'enseignement du français : Les caractères accentués (é, è, ê, ë, à, â, ç, œ, æ, î, ï, ô, û, ü) sont codés sur 2 octets en UTF-8. Un fichier texte contenant du français doit être enregistré en UTF-8 pour éviter les problèmes d'affichage (apparition de « ?? » ou de caractères illisibles).

5. Codage des images bitmap

Schéma d'une image bitmap

```
graph TB
  subgraph "Image bitmap 4x4"
    P11["(255,0,0)"]
    P12["(255,0,0)"]
    P13["(0,0,255)"]
    P14["(0,0,255)"]
  end
```

```

P21["(255,0,0)"] P22["(255,0,0)"] P23["(0,0,255)"] P24["(0,0,255)"]
P31["(0,255,0)"] P32["(0,255,0)"] P33["(255,255,0)"] P34["(255,255,0)"]
P41["(0,255,0)"] P42["(0,255,0)"] P43["(255,255,0)"] P44["(255,255,0)"]
end

subgraph "Légende"
R["Rouge = (255,0,0)<br/>RVB 24 bits"]
B["Bleu = (0,0,255)"]
G["Vert = (0,255,0)"]
J["Jaune = (255,255,0)"]
end

style R fill:#ff0000,color:#fff
style B fill:#0000ff,color:#fff
style G fill:#00ff00,color:#000
style J fill:#ffff00,color:#000

```

Pixel = Picture Element. Plus petite unité d'une image numérique.

Résolution : nombre de pixels (largeur × hauteur). Exemple : $1920 \times 1080 = 2\,073\,600$ pixels (Full HD).

Profondeur de couleur : nombre de bits par pixel (bpp).

- 1 bpp : noir et blanc (2 couleurs)
- 8 bpp : 256 couleurs (palette)
- 24 bpp : 16,7 millions de couleurs (RVB : 8 bits par canal)
- 32 bpp : 24 bits + canal alpha (transparence)

Calcul de la taille d'une image

Taille (en octets) = Largeur × Hauteur × (Profondeur / 8)

Exemple : image 1920×1080 en 24 bits
 Taille = $1920 \times 1080 \times 3$
 = 6 220 800 octets
 ≈ 5,93 Mo

Exercice : Une image de 800×600 pixels en noir et blanc (1 bit/pixel) pèse :

$800 \times 600 \times 1/8 = 60\,000$ octets ≈ 58,6 Ko

Comparaison : Une photo en 24 bits de 12 mégapixels (4000×3000) pèse :

$4000 \times 3000 \times 3 = 36\,000\,000$ octets = 36 Mo (non compressé)

C'est pourquoi les formats compressés (JPEG, PNG) sont utilisés.

QCM d'auto-évaluation

- Combien de bits y a-t-il dans un octet ?
 a) 4 bits b) 8 bits ✓ c) 16 bits
- Quelle est la valeur décimale de 1010_2 ?
 a) 8 b) 10 ✓ c) 12
- Quel encodage de caractères est dominant sur le Web ?
 a) ASCII b) UTF-8 ✓ c) ISO-8859-1

4. Combien d'octets utilise UTF-8 pour coder le caractère « é » ?
a) 1 octet b) 2 octets ✓ c) 3 octets
5. Une image de 1024 × 768 pixels en 24 bits pèse approximativement :
a) 768 Ko b) 2,25 Mo ✓ c) 5 Mo

Conclusion

La représentation de l'information est le fondement de toute l'informatique. Les conversions binaire/hexa/décimal, le codage des caractères (ASCII, Unicode/UTF-8) et le codage des images sont des notions essentielles pour comprendre comment un ordinateur stocke et manipule les données. Pour un enseignant de français, la maîtrise du codage UTF-8 est particulièrement importante pour éviter les problèmes d'affichage des caractères accentués dans les documents numériques.

Prochaine séance : Architecture machine — le modèle de von Neumann et le cycle d'exécution.

Note pédagogique ENS

Transposition didactique : En classe de SNT Seconde (thème « Données structurées »), on peut proposer :

- Activité débranchée : coder son prénom en binaire avec des cartes noires (0) et blanches (1)
- Activité : convertir des mots français en ASCII, puis constater que les accents ne sont pas codés
- Activité : créer une image bitmap en noir et blanc sur papier quadrillé (chaque case = un pixel)

Lien avec les langues : Expliquer pourquoi certains caractères français (œ, æ, Œ, Æ) peuvent poser problème dans des fichiers exportés depuis Word vers un autre logiciel (problème d'encodage). Montrer l'importance de toujours enregistrer en UTF-8.

Bibliographie

- Goupille, P. (2014). Technologie des ordinateurs. Éditions Dunod (chapitres 1 et 2).
- Tanenbaum, A. & Austin, T. (2012). Architecture de l'ordinateur (6e éd.). Pearson.
- Unicode Consortium (2024). The Unicode Standard — <https://unicode.org>
- Dowek, G. et al. (2019). SNT Seconde — Sciences Numériques et Technologie. Hachette.
- Wikipedia — « Codage des caractères », « Image bitmap », « UTF-8 » (consultation libre).